# Primary Index / Clustering Index / Dense-Sparse / Multi-Level Index

## 1. Overview

When a data file is stored in **sorted sequential order**, an index can be built on that ordering field to speed up access.

This is called a **Primary Index** in classical DBMS terminology.

```
None
Sorted Data File

+ Index on sorting key

= Primary Index
```

## 2. Important Terminology Note

Many people say:

```
None
Primary Index = Index on Primary Key
```

But classical DBMS meaning is different.

Correct traditional meaning:

```
None
Primary Index = index on the field that determines physical sequential order of file
```

That field may be:

- Primary key
- Candidate key
- Non-key field

# 3. What Is Clustering Index?

If records are physically grouped/sorted by a **non-unique field**, it is often called **Clustering Index**.

Example:

```
None
Employees sorted by department
```

Same department rows stored together.

# 4. Example Data File

Employees table sorted by department:

| EmpID | Name | Dept |
|---|---|---|
| 101 | A | HR |
| 102 | B | HR |

| | | |
|---|---|---|
| 103 | C | IT |
| 104 | D | IT |
| 105 | E | Sales |

# 5. Index Example

```
None
HR    → first HR row
IT    → first IT row
Sales → first Sales row
```

Then rows of same department follow sequentially.

# 6. Dense Index

# Definition

Contains an index entry for **every search-key value**.

```
None
Every key has pointer
```

# Example

Users:

| id |
| --- |
| 1 |
| 2 |
| 3 |

Dense index:

```
None
1 → row1

2 → row2

3 → row3
```

## Advantages

Fast lookup
Direct access

## Disadvantages

More storage
More update cost

# 7. Sparse Index

## Definition

Entries exist for only **some keys**, usually one per block/page.

```
Not every key indexed
```

## Example

Block storage:

```
Block1: 1,2,3

Block2: 4,5,6

Block3: 7,8,9
```

Sparse index:

```
None
1 → Block1

4 → Block2

7 → Block3
```

To find key=5:

- Use index → Block2
- Search inside block

# Advantages

Less storage
Smaller index
Faster maintenance

# Disadvantages

Slightly slower than dense index

# 8. Primary Index on Key Attribute

If file sorted by primary key:

```
None
CustomerID ascending
```

Then often sparse index used.

```
None
One index entry per block
```

# 9. Primary / Clustering Index on Non-Key Attribute

If file sorted by repeated field:

```
None
department

city

category

status
```

Then index usually maps each unique value to first block/record.

Example:

```
None
IT → first IT employee

HR → first HR employee
```

# 10. Why Called Clustering

Rows with same value physically cluster together.

Useful for:

- Department reports
- Region data
- Product category queries

# 11. Multi-Level Index

# Problem

Large single index also becomes slow.

# Solution

Create index on top of index.

```
None
Level 2 Index

   ↓

Level 1 Index

   ↓

Data Blocks
```

# Example

```
None
Master index points to lower indexes

Lower index points to records
```

# Benefits

Fewer disk reads
Scalable indexing

This concept leads to **B+ Trees**, used in modern DBs.

Examples:

- PostgreSQL
- MySQL

# 12. Dense vs Sparse Comparison

| Feature | Dense | Sparse |
| --- | --- | --- |
| Entry for every key | Yes | No |
| Storage | High | Low |
| Search speed | Faster | Slightly slower |

| | | |
|---|---|---|
| Maintenance | More costly | Cheaper |

# 13. Primary vs Secondary Index

| Type | Meaning |
|---|---|
| Primary Index | Based on physical sorted order |
| Secondary Index | On other columns |

# 14. Real HLD Perspective

Modern systems rarely discuss “primary index” in old textbook language.

Instead discuss:

- Clustered index
- B+ Tree index
- Secondary index
- Composite index
- Partition key

Still useful for fundamentals.

# 15. Interview Answer Example

**Q: What is sparse index?**

> Sparse index stores entries for selected keys (often one per block), reducing storage while still allowing fast lookup using block-level pointers.

# 16. Short Revision Notes

```
Primary Index:

index on sorting field of file


Dense:

every key indexed


Sparse:

some keys indexed


Multi-level:

index on index
```

# 17. One-Line Summary

**Primary indexing uses the file's physical sort order to speed access, while dense, sparse, and multi-level indexes trade storage and lookup speed to efficiently scale searches.**